

1. Hardware Platform & Electrical System

The physical and electrical architecture of the vehicle is designed to sustain high-speed autonomous racing operations, requiring high-bandwidth data acquisition, rapid computational throughput, and agile mechanical actuation.

가. Lower Level Chassis & Mechanical Parameters

The vehicle utilizes the Traxxas Slash 4x4 VXL as its foundational mechanical platform, providing a robust four-wheel-drive suspension layout. Propulsion is delivered via a high-performance 4-pole sensored brushless DC motor (XERUN 4268SD, 1900KV), capable of handling high peak discharge conditions during rapid acceleration. Key geometric and physical parameters dictate the constraints of the vehicle dynamics model.

Parameter / Item	Specification	Application / Reference
Base Chassis Model	Traxxas Slash 4x4 VXL	Mechanical platform configuration
Drive Motor	XERUN 4268SD (Sensored BLDC, 1900KV)	4-pole high-torque racing motor
Wheelbase (L)	0.324 m	Kinematic & dynamic steering geometry
Maximum Steering Angle	$\pm 23.4^\circ$ (0.4089 rad)	Servo motor physical steering limits

나. Computing & Electronic Control Units

Centralized compute tasks are performed on an embedded GPU-accelerated system running Ubuntu Linux with the Robot Operating System (ROS 2). Low-level motor commutation, encoder odometry execution, and PWM servo signaling are routed via an advanced open-source Electronic Speed Controller (ESC).

Component	Product / Operating System	Technical Profile	Interface Port
On-Board Computer	NVIDIA Jetson Orin NX	Ubuntu 22.04 LTS / ROS 2 Humble	Primary Compute Unit
Motor Controller	VESC mkVi	Firmware Configured (V 5.0.2)	Serial (/dev/ttyACM0)
Main Battery Pack	Lithium-Polymer (Li-Po)	4S configuration, 5250mAh	Power Distribution Board

다. Autonomous Perception Sensor Suite

To ensure accurate state estimation and localized track boundary recognition, the platform integrates a complementary mix of range-finding, inertial, and wheel odometry sensors.

Sensor Type	Model Name	Hardware Interface	Functional Parameters & Role
LiDAR	Hokuyo UST-30LX	Ethernet Protocol	Range: 30m, Scan: 270°, Res: 0.25°
IMU	3DM-CV7-AHRS	USB Connection	9-DOF, High-precision angular velocity & linear acceleration
Encoder	Built-in Hall Sensors	Internal Bus (to USB)	Motor RPM tracking for odometry calculation

라. Power Consumption Profiling

Power budgeting is critical to guarantee voltage stability across the computing unit during peak acceleration cycles. The system's power consumption profile is highly dynamic, fluctuating heavily based on traction load.

Subsystem / Hardware Item	Power Consumption (W)	Operational Behavior
XERUN 4268SD	600 W - 1,000 W (Continuous) 1,800 W - 2,500 W (Peak)	Main thermal and electrical load source
AX2255 Steering Servo	20 W (Rated) / 35 W (Stall Condition)	Highly dependent on dynamic lateral tire forces
NVIDIA Jetson Orin NX	20 W - 25 W	Scales directly with GPU/CPU core usage
Hokuyo UST-30LX LiDAR	1.8 W	Constant power dissipation profile
3DM-CV7-AHRS IMU	0.28 W (280 mW)	Negligible draw profile
Complete Vehicle Assembly	650 W - 1,900 W (Average Operating Range)	Requires rigorous C-rate safety margins

2. 2. Control Systems Architecture & Algorithmic Structure

The vehicle utilizes a Smooth Model Predictive Path Integral (SMPPI) control framework. SMPPI is an optimization-driven approach designed to discover optimal vehicle trajectories under highly non-linear, transient tire-slip conditions.

가. Mathematical Objective Function

The foundational core of the SMPPI controller is to minimize the expected cumulative cost over a finite prediction horizon (T). The optimal control sequence (U) is computed by optimizing the expectation over stochastically sampled variations:

$$\min_U E \left[\sum_{t=0}^{T-1} C_{\text{running}}(x_t, u_t) + C_{\text{terminal}}(x_T) \right]$$

4. Cost Function Component Matrix

To prevent myopic behavior and dictate optimal high-speed maneuvers, the cost matrix evaluates both transient safety indices and overall path progress.

1. Running Cost (J_{run}): Enforces trajectory smoothing and boundaries. A control smoothness penalty (steer_cost) limits high-frequency oscillations. Heading errors compared to the center-line are penalized (vel_cost). Crucially, a Softplus-based boundary_cost penalizes proximity to walls, acting as a non-linear soft barrier function to eliminate trajectory erosion near borders.

2. Terminal Cost (J_{term}): Rewards global performance metrics evaluated at the horizon boundary (T). It penalizes lack of track index displacement (progress) and rewards the exit velocity square (escape_vel). This combination naturally incentivizes the vehicle to discover and execute classic Out-In-Out racing lines without requiring predefined reference path coordinates.

5. Dynamic Models and Non-linear Hard Constraints

To handle transitions from standstill conditions to high-speed slip-limited regimes, the system operates as a hybrid formulation:

1. Kinematic Regime ($v < 0.5$ m/s): Implements a standard kinematic bicycle model to sidestep geometric singularities caused by near-zero denominators in slip angle computations.

2. Dynamic Regime ($v \geq 0.5$ m/s): Switches to a non-linear dynamic configuration integrating vehicle mass, yaw inertia, and motor back-EMF damping coefficients. Lateral and longitudinal forces are governed by empirical Pacejka Magic Formula tire models, parameterized by front/rear stiffness coefficients.

3. Cut-off & Evaluation Constraints: Paths exceeding lateral threshold limits ($>1.3G$ lateral acceleration) or violating maximum actuator slew rates are immediately assigned a catastrophic penalty value and removed from the sampling pool.

6. Software Implementation Pipeline & Parallelization

Computing thousands of stochastically perturbed trajectory forward simulations

(rollouts) within a fraction of a control loop requires hardware-level parallel execution. The package architecture is partitioned as follows:

1. CUDA Parallel Processing

가) Resolves computational bottlenecks by offloading high-density calculations to custom GPU kernel functions.

나) Manages the parallel execution of trajectory rollouts, dynamic state updates, and cumulative cost evaluations across thousands of sampled scenarios.

다) Executes the primary stochastic optimization routine directly within the GPU memory space.

2. ROS 2 System Integration

가) Integrates the optimized CUDA-based MPPI solver within a standard ROS 2 node architecture.

나) Subscribes to vehicle localization data and odometry feedback from both simulation environments and the physical vehicle.

다) Triggers the asynchronous GPU computation block to ensure deterministic control loop execution.

라) Publishes the computed optimal linear velocity and steering targets via standard AckermannDriveStamped messages to actuate the platform.

마. Localization & State Estimation

Accurate vehicle localization is essential for autonomous racing, as the trajectory optimization framework requires reliable estimates of vehicle pose and motion states. The platform employs a map-based probabilistic localization framework utilizing LiDAR measurements and wheel odometry to continuously estimate the vehicle position within the racing environment.

1. Reference Map Generation

Prior to autonomous operation, a two-dimensional occupancy grid map of the driving environment is generated using SLAM Toolbox. The package performs Simultaneous Localization and Mapping (SLAM) based on LiDAR scan

measurements, enabling the construction of a geometrically consistent representation of track boundaries and surrounding structures. The generated occupancy grid map is subsequently stored and utilized as a reference map during autonomous operation.

2. Adaptive Monte Carlo Localization (AMCL)

Vehicle localization is performed using Adaptive Monte Carlo Localization (AMCL), a particle filter-based probabilistic localization algorithm. AMCL continuously estimates the vehicle pose by comparing real-time LiDAR observations against the reference occupancy grid map.

A population of weighted particles is maintained, where each particle represents a potential vehicle pose hypothesis. Particle weights are updated according to the likelihood of observed LiDAR scans, followed by iterative resampling procedures that concentrate computational resources around highly probable vehicle locations.

The adaptive sampling mechanism dynamically adjusts the particle population according to localization uncertainty, improving computational efficiency while maintaining robust localization performance during dynamic driving conditions.

3. State Estimation Integration

The estimated vehicle pose is combined with wheel encoder odometry and inertial measurements to provide a complete vehicle state representation for the control framework. Localization outputs are continuously published through ROS 2 topics and consumed by the SMPPI controller for trajectory prediction and optimization.

The resulting state estimate provides global position, heading angle, linear velocity, and angular velocity information required for closed-loop autonomous vehicle control.

4. Dynamic Obstacle Detection & Tracking

Reliable detection and tracking of dynamic obstacles are essential for maintaining safe autonomous operation in high-speed racing environments. The proposed perception framework utilizes 2D LiDAR measurements to accurately identify, classify, and track moving objects located ahead of the ego vehicle. By combining geometric feature extraction, map-based spatial filtering, and non-linear state estimation techniques, the system achieves robust obstacle tracking performance under noisy sensing conditions.

1. Point Cloud Processing and Geometric Feature Extraction

Raw 2D LiDAR scan data are segmented using Euclidean clustering to separate individual objects from the surrounding environment. Candidate clusters are filtered according to predefined geometric constraints corresponding to the expected dimensions of target vehicles.

For each valid cluster, an L-Shape Fitting algorithm is applied to estimate the object's rectangular contour and orientation angle. This approach enables reliable extraction of vehicle geometry even when the observed point cloud contains measurement noise, partial occlusions, or sparse returns.

2. Map-Based Spatial Filtering

To prevent false-positive detections caused by track-side structures, barriers, and curved boundary segments, a map-based spatial filtering process is incorporated into the perception pipeline.

The system continuously references a validated track centerline representation generated during the mapping stage. Only objects whose centroid positions lie within a predefined distance threshold from the centerline are considered valid dynamic obstacles. This filtering mechanism significantly reduces the likelihood of incorrectly classifying static environmental structures as moving vehicles.

3. Shape-Aware Non-Linear State Estimation

To maximize tracking accuracy, geometric shape information obtained from L-Shape Fitting is tightly coupled with a non-linear motion estimation framework.

The extracted object contour and orientation measurements are incorporated as observations within an Extended Kalman Filter (EKF). Target vehicle motion is modeled using a Constant Turn Rate and Velocity (CTRV) dynamic model, enabling robust state prediction during aggressive cornering maneuvers and transient vehicle motions.

The tracking framework continuously estimates the target vehicle position, velocity, heading angle, and turn rate while compensating for sensor noise and geometric distortions. During data association, conservative matching thresholds are employed to minimize identity switching and prevent track corruption caused by nearby walls or spurious measurements.

The resulting state vector of each tracked obstacle is transmitted to the vehicle control system, where it is incorporated into trajectory evaluation and collision avoidance processes.

4. System Output

The final tracking output provides a stable and continuously updated estimate of dynamic obstacle states, including position, velocity, heading, and predicted motion. These estimates are directly utilized by the SMPPI controller to generate safe and dynamically feasible avoidance trajectories during autonomous operation.